

University Of Potsdam
Informatik/Computational Science

Bachelor's Thesis

Intelligent Malfunction Handling in Large Railway Systems

Author:
Iannis Valentin Waldow

Supervisors:
Ryan Kepler James Murphy,
Prof. Dr. Torsten Schaub

A thesis submitted in fulfillment of the requirements for the degree of Bachelor of Science (B.Sc.)

in the

Knowledge Representation and Reasoning Group
at the Faculty of Computer Science

December 7, 2025

Abstract

Improving the resilience of modern railway systems against unforeseen disruptions remains a pressing issue. The underlying vehicle rescheduling problem (VRSP) has been the focus of considerable research effort. In 2020, a research group involving members of the Swiss and German railways presented a novel machine learning approach to the VRSP in "Flatland-RL: Multi-Agent Reinforcement Learning on Trains". In Flatland environments, trains employ reinforcement learning to navigate to their destinations autonomously.
The Knowledge Representation and Reasoning Group at the University of Potsdam (KRR-UP) has been conducting research into using the environments provided by Flatland as a basis to approach the VRSP using Answer Set Programming (ASP). Recent work has made progress in addressing malfunction scenarios. However, a standardized, extensible framework for handling malfunctions is still lacking. This paper introduces such a framework, to enable further research in malfunction handling using ASP.

Kurzfassung

Die Verbesserung der Resilienz von modernen Eisenbahnsystemen gegen unvorhergesehene Störungen bleibt ein dringendes Problem. Das zugrundeliegende Fahrzeugumplanungsproblem (VRSP) ist Gegenstand erheblichen Forschungsaufwands. Im Jahr 2020 stellte eine Forschungsgruppe, an der Mitglieder der Schweizer und Deutscher Bahn beteiligt waren, einen neuartigen Machine-Learning-Ansatz zum VRSP in "Flatland-RL: Multi-Agent Reinforcement Learning on Trains" vor. In Flatland-Umgebungen nutzen Züge Reinforcement-Learning, um autonom zu ihren Zielen zu navigieren.
Die "Knowledge Representation and Reasoning Group" an der Universität Potsdam (KRR-UP) forscht aktiv, die von Flatland bereitgestellten Umgebungen zu nutzen, um mittels ASP das VRSP anzugehen. Jüngste Arbeiten in der Gruppe haben bereits Fortschritte in der Behandlung von Störungen gemacht. Jedoch bedarf es noch eines standardisierten, erweiterbaren Rahmenwerks für den Umgang mit Störungen. Diese Arbeit präsentiert ein solches Rahmenwerk, um weitere Forschungsarbeit im Bereich der Störungsbehandlung mittels ASP zu ermöglichen.

Contents

1 Introduction **4**
- 1.1 Flatland . . . 4
  - 1.1.1 Overview . . . 4
  - 1.1.2 Actions and Plans . . . 5
  - 1.1.3 Example Flatland Environments . . . 6
- 1.2 ASP . . . 7
  - 1.2.1 Environment Facts . . . 7
  - 1.2.2 Action Facts . . . 7
  - 1.2.3 Encodings . . . 7
  - 1.2.4 Workflow between Flatland and ASP . . . 8

2 Problem Discussion **9**
- 2.1 VRSP . . . 9
- 2.2 Collisions . . . 9
- 2.3 Valid Plans . . . 10
- 2.4 Example . . . 11

3 Malfunctions **12**
- 3.1 Malfunctions in Flatland . . . 12
  - 3.1.1 Flatland's Stochastic Malfunction Model . . . 12
- 3.2 Malfunctions in ASP . . . 12
- 3.3 Malfunction Response Framework . . . 13
  - 3.3.1 Step 1: Determine Triviality . . . 13
  - 3.3.2 Step 2: Replanning . . . 13
  - 3.3.3 Integration into the Established Workflow . . . 14
  - 3.3.4 Making Agents Wait . . . 14

4 Triviality **15**
- 4.1 Motivation . . . 15
- 4.2 Conflict Points . . . 16
- 4.3 Deltas . . . 16
  - 4.3.1 Relative Delta . . . 17
- 4.4 Determining Triviality Using Deltas . . . 17
- 4.5 Examples . . . 21
  - 4.5.1 First Example . . . 21
  - 4.5.2 Second Example . . . 21

5 Replanning **23**
- 5.1 Naive Replanning . . . 23
- 5.2 Redo Replanning . . . 24
- 5.3 Wait Replanning . . . 24
- 5.4 Replanning Requirements . . . 25
  - 5.4.1 Hard Requirements . . . 25
  - 5.4.2 Soft Requirements . . . 25

6 Implementation **26**
6.1 Python Framework 26
6.1.1 Overview 26
6.1.2 ASP Knowledge Base 26
6.1.3 Property Agent 26
6.2 ASP Encoding 27
6.2.1 Triviality 27
6.2.2 Deltas 28
6.2.3 Replanning 28
6.2.4 Soft Requirements 28

7 Discussion **29**
7.1 Machine Learning and ASP 29
7.2 Concerning Replanning 29
7.2.1 Defining a Scope 29
7.2.2 Reducing the Search Space 29

8 Conclusion **30**
8.1 Further Work 30

1 Introduction

In this paper we introduce a response framework to malfunctions in a simulated railway environment using ASP. The railway simulation environment we use is "Flatland" [1], initially introduced in 2020 to research machine learning and reinforcement learning approaches to the vehicle scheduling problem (VSP) and the vehicle rescheduling problem (VRSP). Flatland has been the basis for multiple benchmarks and competitions pushing the limits of reinforcement learning in multi-agent path finding problems[2, 3, 4].

In a Flatland environment our goal is to find valid routes for all trains to reach their destination. During the simulation malfunctions can randomly occur, which force a train to stop.

We will start with an overview of Flatland environments, agents and plans. Following that, we will discuss the problem of vehicle rescheduling in more detail and introduce the response framework and its concepts of triviality and rescheduling.

1.1 Flatland

1.1.1 Overview

A Flatland environment is a grid of cells, containing tracks, stations and trains. In this paper we will usually refer to the trains in Flatland environments as 'agents'.

Definition 1 (*Flatland Environment*)
We define a Flatland environment Env of any width $x \geq 1$ and height $y \geq 1$ as the tuple

$$Env = (Cells, Agents),$$

where
$Cells \subseteq (\{\{1, \ldots, x\} \times \{1, \ldots, y\}\}) \times Types$;
$Agents \subseteq \{(start, end) \mid start, end \in Cells\}$, where $start$ and end denote start and end point respectively;
$Types$ is a given set of numbers, encoding all types of cells in Flatland.

The cells of the grid can either be empty or contain one of 7 types of tracks. All track types can be rotated. If a cell contains a track, it can be a start or end point for an agent.

Orientation is an important concept within Flatland, since all agents face either north, east, south or west. Agents cannot reverse, meaning they have to go down the track they are on in the direction they are facing.

The end points of agents, or destinations, are graphically represented as stations in Flatland environments.

The simulation of a Flatland environment keeps track of time in discrete steps, with the simulation starting at step 0. At every step, the simulation executes all actions provided. If there are no actions provided for an agent it waits. The simulation ends when there are no more actions to execute or when all agents have reached their destination.

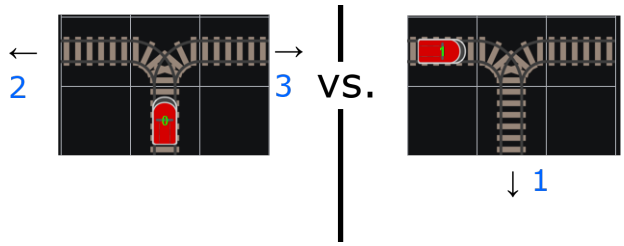


Figure 1: Actions (blue numbers) an agent can take when entering a fork. Note, that the agent would need to be *on* the fork to execute the action[1].

1.1.2 Actions and Plans

There are 4 actions an agent can take:

- 0: Wait
- 1: Go forward
- 2: Go left
- 3: Go right

The numbers correspond to the internal representation used within Flatland.

While an agent can always wait, the three movement actions can only be taken depending on the agent's current position. If there is a fork going left and right for example an agent can chose either action 2 or 3, but not 1. If the agent enters the fork from another direction, it is quite possible that the movement options available are different (refer to fig. 1).

Since agents usually have to take multiple steps to reach their destinations, it will not do to provide just a single movement command. We have to come up with a plan for every agent.

Definition 2 (*Plan*)
A plan is a sequence of $n \in \mathbb{N}$ actions of the set $Actions = \{0, 1, 2, 3\}$ that an agent executes to move through an environment.

$$(p_i)_{i=0}^{n}$$

where $p_i \in Actions$ for all $0 \leq i \leq n$.

In our notation, when distinguishing between different plans we will refer to the plan of an agent a by p^a. When p^a is passed as an input in a definition, we implicitly expect a to also be passed as an input.

For convenience we will also denote the length of an agent's plan p^a to be $|p^a|$. This means the actions in the sequence can be accesses with $p^a_0, p^a_1, \ldots, p^a_{|p^a|}$.

We can simulate a plan in the Flatland environment, which updates the agent's position. To more easily keep track of where agents end up after taking a given number of steps, we define the position function.

[1] Graphics courtesy of the Clingonia Project



Figure 2: A Flatland environment

Definition 3 (*Position Function*)
The position function returns the cell c of environment $e = (Cells, Agents)$, that an agent $a = (start, end) \in Agents$ will be in after taking t steps of its plan p^a from its starting point $start$.

$$pos(a, p^a, t) = c$$

Note, that the definitions of the position function and agent plans p^a need to know about the environment e they are working in. Wanting to keep notation clean, we opt to have knowledge of the environment be implicit. Going forward, if an environment is given as part of a definition, we implicitly expect what is being defined to work within the context of that environment.

Combining our definitions for agent plans with the position function, this means the last cell an agent a will reach is $c = pos(a, p^a, |p^a|)$.

1.1.3 Example Flatland Environments

Figure 2 shows an environment in Flatland.

On the tracks we can see three agents: One red, one purple and one pink. We can see the destinations of all agents as stations. Which station each agent is trying to reach can be determined by looking at the line underneath each station, its color corresponding to that of one of the agents.

In this paper we will be making use of the *handmade* environment seen in fig. 2, to more easily reason over.

1.2 ASP

We use Answer Set Programming (ASP) [5] to encode Flatland environments and to perform all of the reasoning discussed in this paper.

For grounding and solving we use "clingo"[6], which includes the grounder "gringo" and solver "clasp".

In order to work with the Flatland environments in ASP, they first need to be translated to an ASP fact format. Once an environment is in ASP's fact format, we can define rules to come up with plans and solutions. More detail on this follows in sections 1.2.3 and 2.

As part of the preceding work[2] on Flatland and ASP at the Knowledge Representation and Reasoning Group at the University of Potsdam (KRR-UP), a standard for how to present Flatland environments and actions within ASP has been agreed upon.

1.2.1 Environment Facts

There are two types of environment facts. *Cell facts* and *agent facts*. The prior has this shape:

```
cell((X,Y), Type).
```

`(X,Y)` provide the coordinate of the cell. `Type` sets the cell type as an integer. A cell of type 0 is an empty cell and can be ignored, as agents should not end up on them.

There are three agent facts relevant to us:

```
% Existence Fact
train(Z).
% Start Point (X,Y) of train Z and earliest departure D
start(Z,(X,Y),D,H).
% End Point (X,Y) of train Z and latest arrival A
end(Z,(X,Y),A).
```

A fourth agent fact for setting train speed exists, but is outside the scope of this paper.

1.2.2 Action Facts

Action facts are returned by an encoding after solving.

```
action(train(ID), Action, Time).
```

If we want to simulate these actions in the Flatland environment, we first need to translate them back to a format that can be read by Flatland.

1.2.3 Encodings

An encoding takes the environment facts and defines a logic with which it will look for solutions. The underlying logical structure can differ, from simply moving the agents through the environment directly to constructing a graph of the environment first before looking for paths.

[2]Refer to the KRR-UP GitHub project page

1.2.4 Workflow between Flatland and ASP

Flatland is available as a Python library[3], which is how we interface with its environments.

Together with Clingo's Python library, the general workflow between ASP and Flatland looks as follows.

1. **Build** the environment (using '`build.py`')
    - We build the environment, either manually or procedurally
    - This step returns the environment in two file formats,
        - Python readable (`pickle`)
        - the equivalent translation into ASP (`lp`)
2. **Solve** the environment (using '`solve.py`')
    - To solve, we pass in both environment files and an encoding
    - ASP uses the environment facts and encoding to find a solution
    - ASP returns a solution. If none is found, the program **terminates**
    - The solution in ASP fact format is translated back for Flatland
3. **Simulate** the environment (usually done as part of '`solve.py`')
    - Flatland simulates the solution visually
    - Once all actions are simulated the program **terminates**

[3]Refer to the Python library page of flatland-rl

2 Problem Discussion

So far we have covered all the parts of Flatland environments that are relevant to us and briefly looked at how they can represented within ASP. Let us now take a closer look at the problem of getting all agents to their destination, the vehicle scheduling problem, and its extension, the vehicle *rescheduling* problem.

2.1 VRSP

In the context of a Flatland environment, the objective of the vehicle scheduling problem is getting all agents to their destination station. We often refer to this to as 'solving' the environment. In order to do so, all agents have to be issued one action per simulation step until they have all reached their destination.

The vehicle rescheduling problem (VRSP) extends the classic vehicle scheduling problem (VSP) by introducing unpredictable, stochastic events that cause delays. In railway operation this could be anything from track work to technical issues or medical emergencies. In Flatland such events are modeled as "malfunctions".

In the literature, we can see the VRSP formalized in terms of serving multiple customers on a train's trip[7] or using depots to fill gaps in operation[8]. In the context of Flatland the focus stays firmly on getting agents from starting point to ending point. Trips cannot be cancelled, nor can new trips be introduced.

Our approach to the VRSP can be broadly separated into the discussion of two problems: Determining malfunction triviality and replanning. Both play an important role in our overall malfunction response framework and will be examined in greater detail in their own sections later on.

2.2 Collisions

Depending on the agent plans provided, it is possible we encounter collisions.

There are two types of collision that can take place in an environment. The first happens when two agents try to share the same cell at the same time.

Definition 4 (*Type 1 Collision Set: Cell Collisions*)
Given an environment $e \in Env = (Cells, Agents)$ and two plans p^{a_1}, p^{a_2}, the set Col_1 contains all steps at which cell collisions occur between two *different* agents a_1 and a_2.

$$Col_1(p^{a_1}, p^{a_2}) = \{t \mid pos(a_1, p^{a_1}, t) = pos(a_2, p^{a_2}, t)\}$$

Cells themselves are not the only places where collisions can occur. Agents can also collide on edges, the space between two cells of the environment. Because of the discrete representation of time and space in Flatland, with edges not being logical units of space, the same *unit* of space is never shared at the same time when two agents pass through an edge simultaneously.

If edge collisions were not considered, two directly opposing agents could swap places by simply moving *through* one another. They would never need to share the same cell at any point to do so, because the collision happens somewhere in between the discrete steps of the simulation, on the edge.

In order to catch edge collision, let us first introduce edges as a logical unit of space, defined as a set of two cells $\{c_1, c_2\}$, and extend the position function of definition 3 to include edges as an output.

Definition 5 (*Extended Position Function*)
The extended position function returns a set containing the cell c_2 of environment $e = (Cells, Agents)$ an agent $a \in Agents$ will be in after taking t steps of a given plan p^a. If the agent has moved onto the cell from another cell c_1, the edge $\{c_1, c_2\}$ is also part of the output set.

$$\begin{aligned} Pos_e(a, p^a, t+1) = &\{pos(a, p^a, t+1)\} \cup \\ &\{c_1, c_2 \mid c_1 = pos(a, p^a, t), c_2 = pos(a, p^a, t+1), \\ &\quad c_1 \neq c_2\} \end{aligned}$$

It is impossible for an agent to travel to an edge directly, unlike how it would to a cell. Instead an agent travels to an edge cell automatically at the step it passes over that edge. At that step its position is logically in both a cell and an edge.

Now that we have agents logically occupying edges, we can define the second type of collision type, edge collisions.

Definition 6 (*Type 2 Collision Set: Edge Collisions*)
Given an environment $e \in Env = (Cells, Agents)$ and two plans p^{a_1}, p^{a_2}, the set Col_2 contains all steps at which edge collisions occur between two *different* agents a_1 and a_2.

$$\begin{aligned} Col_2(p^{a_1}, p^{a_2}) = \{t \mid &\{c_1, c_2\} \in Pos_e(a_1, p^{a_1}, t), \\ &\{c_1, c_2\} \in Pos_e(a_2, p^{a_2}, t)\} \end{aligned}$$

Since the extended position function outputs both cell and edge positions we can use it more broadly to determine the set of both collision types. The complete collision set can also be formed as a union of the sets for collision type one and two.

Definition 7 (*Complete Collision Set*)
The set of all collisions between two plans p^{a_1}, p^{a_2} is

$$\begin{aligned} Col(p^{a_1}, p^{a_2}) = \{(a_1, a_2, t) \mid &c \in Pos_e(a_1, p^{a_1}, t), \\ &c \in Pos_e(a_2, p^{a_2}, t)\} \end{aligned}$$

Alternatively, we can also take the complete collision set as the union of Col_1 and Col_2:

$$Col(p^{a_1}, p^{a_2}) = Col_1(p^{a_1}, p^{a_2}) \cup Col_2(p^{a_1}, p^{a_2})$$

2.3 Valid Plans

In order to solve the VSP and thus the environment, we have to come up with valid plans for the environment.

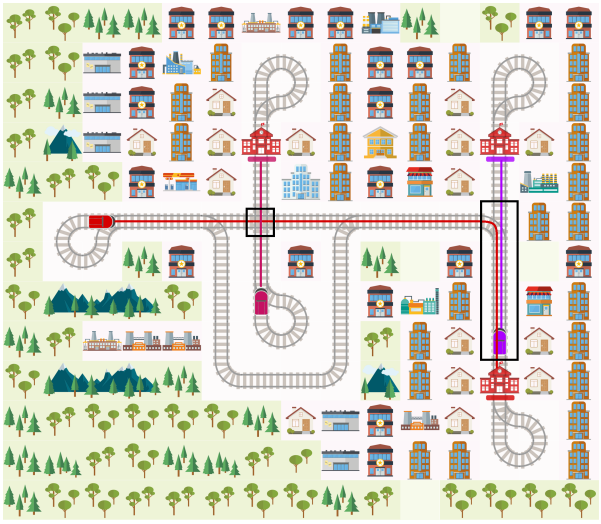


Figure 3: Plan shown in Flatland environment

Definition 8 (*Agent Solution*)
An agent solution is a plan p^a, which, if executed, gets the agent $a = (start, end)$ to its destination. Formally, this plan p^a has to fulfill

$$Pos(a, p^a, |p^a|) = end.$$

If our environment only contained one agent, this would be the end of it. For environments with more than one agent we need to make sure that the solutions of all agents are not mutually conflicting.

Definition 9 (*Environment Solution*)
An environment solution P_{Agents} is a set that contains an agent solution for every agent in the environment e.

All solutions within the set have to be mutually collision free,

$$\forall p^{a_1} \in P_{Agents}. \forall p^{a_2} \in P_{Agents} : Col(p^{a_1}, p^{a_2}) = \emptyset.$$

2.4 Example

Let us look at our example from section 1.1.3 again. A possible solution to the VSP posed within could be the one shown in figure 3. The solution has all agents go forward until they have reached their destination:

$$\begin{aligned} p^{a_{red}} &= (1,1,1,1,1,1,1,1,1,1,1,1,1,1), \\ p^{a_{purple}} &= (1,1,1,1), \\ p^{a_{pink}} &= (1,1,1,1,1) \end{aligned}$$

In figure 3 we see the plans as colored lines drawn onto the tracks. The cells in which the plans of any two agents overlap are framed in black.

3 Malfunctions

3.1 Malfunctions in Flatland

A malfunction in Flatland is an event that can happen to an agent at random. We do not know in advance when or for which agent a malfunction will take place. When a malfunction does take place, we do however know for how long it will last. The malfunctioning agent is then prohibited from moving for the duration of the malfunction.

3.1.1 Flatland's Stochastic Malfunction Model

The Flatland Python library allows us to set a malfunction rate for the agents.

In order to set up malfunctions, this data structure must be passed on to the environment during creation:

```
stochastic_data = {
    'prop_malfunction': 0.5,  # Percentage of defective agents
    'malfunction_rate': 30,  # Rate of malfunction occurrence
    'min_duration': 3,  # Minimal duration of malfunction
    'max_duration': 10  # Max duration of malfunction
}
```

`'prop_malfunction'` sets the proportion of agents that are able to experience malfunctions. Above, this means that half of all trains can malfunction.

`'malfunction_rate'` sets the average number of steps an agent takes to experience a malfunction. This is modelled as a poisson process. The malfunction rate equals the average of this poisson process.

`'min_duration'` and `'max_duration'` set lower and upper bounds on the length of malfunctions. Values from this range are sampled uniformly when a malfunction's length is picked.

3.2 Malfunctions in ASP

The KRR-UP Flatland project defines a fact format for malfunctions.

```
malfunction(Z,D,T).
```

This fact states that the train with ID `Z` (**Z**ug) started experiencing a malfunction of length `D` (**D**uration) starting at step `T` (**T**ime step). Adapting this format, we can formally define malfunctions as follows.

Definition 10 (*Malfunction*)
Given an environment $e = (Cells, Agents)$, a malfunction is the tuple

$$m = (a, dur, t),$$

where
$a \in Agents$;
$dur \in \mathbb{N}^+$;
$t \in \{0, \ldots, |p^a|\}$

3.3 Malfunction Response Framework

This paper proposes a framework to respond to malfunctions, integrated into the preexisting workflow. It comprises two steps: Triviality checking and replanning. By checking triviality, we determine if replanning is necessary or not. This step is executed for every malfunction that occurs. As a second step, replanning is only executed if we deem the malfunction non-trivial.

In our discussion of this framework we shall often limit our scope of observation to be as small as possible. As we could already see in the definition for environment solutions (def. 9), the validity of a solution for an entire environment can be determined by comparing the compatibility of individual pairs of agent plans. We assume such an environment solution is provided to us. This means we can limit our scope of discussion to just...

1. one malfunction,
2. the malfunctioning agent,
3. one other agent,
4. one possible collision point (conflict point),
5. the part of the environment the agents travel.

The process we apply to this scope can then be repeated for any other combination of value assignments (other agents, conflict point and so on).

The framework responds to one malfunction at a time. It can respond to multiple malfunctions if these happen sequentially. Each time a malfunction occurs, the same the framework can be reused. The framework would need to be adapted to handle multiple malfunctions happening simultaneously.

3.3.1 Step 1: Determine Triviality

The idea behind checking triviality is to determine whether a malfunction leads to a collision. When a malfunction occurs, we check if the malfunction is trivial using a method we will discuss in section 4. In the simplest terms, we are trying to determine if having the malfunctioning agent wait out the malfunction still results in all agent plans being collision free with one another as described in definition 9. Should we find that having the agent wait out the malfunction does not result in encountering a collision, that malfunction is ruled trivial. Otherwise we move on to the second step, replanning.

3.3.2 Step 2: Replanning

If a malfunction leads to a collision when following the same plan as before, replanning is required. The replanning step is responsible for creating a new environment solution. For this there are multiple approaches, which we will discuss in section 5.

3.3.3 Integration into the Established Workflow

In implementation, both the first step of triviality checking and replanning are handled by ASP. To give a better overview of how this framework slots into the established flow, we extend the flow shown in section 1.2.4.

1. **Build** the environment (as before)
2. **Solve** the environment (as before)
3. **Simulate** the environment (using Flatland)
   - Flatland simulates the solution visually
   - (*New*) The simulation possibly generates a malfunction
   - Once all actions are simulated the program **terminates**
4. **Check** triviality (using ASP)
   - If the simulation of step 3 produces a malfunction we check its triviality
   - If trivial, insert agent wait actions for malfunctioning train and return to step 3
   - If *not* trivial, go to step 5
5. **Replan** the solution (using ASP)
   - Using knowledge from the previous plan, come up with a new solution
   - Once a new solution is found, translate it back for Flatland
   - Return with the translated solution to step 3

3.3.4 Making Agents Wait

In order to more clearly define what we mean by 'having an agent wait out a malfunction', we define a wait insert function to help us be more clear going forward.

Definition 11 (*Wait Insert Function*)
The wait insert function $ins(p^a, d, t) \rightarrow p'^a$ inserts d wait commands at position t of a plan p^a and moves back all actions from t by d steps.

$$ins(p^a, d, t) = (p^a_0, \ldots, p^a_{t-1}, \underbrace{0, \ldots, 0}_{d}, p^a_t, \ldots, p^a_{|p^a|})$$

If we take the values dur and t from a malfunction $m = (a, dur, t)$ and pass them on to the wait insert function, $ins(p^a, dur, t)$, the resulting change to our plan is the malfunctioning agent waiting out the malfunction.

We need to define this function, since without it, we would not be keeping our simulation (Flatland) and planning (ASP) in sync. If a malfunction occurs in the Flatland environment, but we do not respond to it in ASP, the old solution remains in effect and we continue to issue movement command to the malfunctioning agent, which will be ignored. This is a behavior we want to avoid. Thus, we insert our malfunction as wait commands on the planning side in response to an unexpected change on the simulation side.

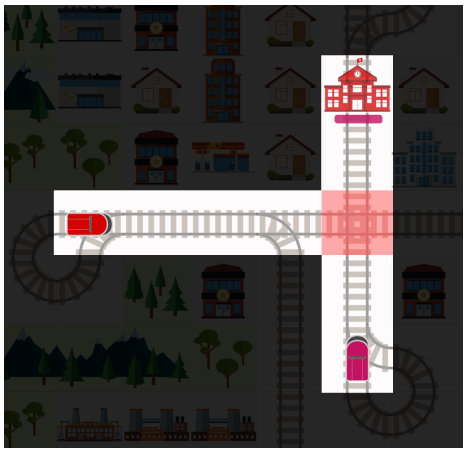


Figure 4: Partial example Flatland environment, with conflict points marked in red

4 Triviality

4.1 Motivation

Before going into and explaining the concepts of triviality and how we will determine it, it is important we first motivate *why* we are trying to determine triviality. Take the example environment of figure 2. And let us limit our scope again to just the red and purple agents. We will look at the part of the environment that is relevant to both, shown in figure 4. The red agent takes 4 steps to reach the red cell, the purple agent takes 2.

Now, let us assume a malfunction occurs for one of these agents. Since replanning can be costly, we would like to avoid it if we can. Hence, we would like to determine whether the malfunction leads to a collision or not.

Say the malfunction happens to the red agent. We can tell, that no collision will result of it, since a delay to the red agent only means it reaches red cell even later than the purple agent does.

Or say the purple agent malfunctions, but at a point at which it has already passed the red cell. In this case we can also clearly see that the malfunction does not cause a collision. The malfunction is 'trivial'.

Definition 12 (*Triviality*)
Triviality is a property that can be ascribed to a malfunction.

A malfunction $m = (a_m, dur, t)$ is considered trivial if we can have the malfunctioning agent a_m wait out the malfunction and continue with its plan from where it was interrupted without causing a collision.

As we see in the example of figure 4, the cell marked in red and the number of steps either agent take to get to it play an important role in determining triviality. We now discuss these two parts, what we call 'conflict points' and 'deltas' and how we can use them to determine if a malfunction is trivial or not.

4.2 Conflict Points

Conflict points are cells or edges in which a collision can occur. Collisions can *only* occur at conflict points. Therefore we are only interested in the effects of a malfunction where it concerns conflict points. Specifically, we want to know if a conflict point could become the site of a collision as an effect of a malfunction.

Definition 13 (*Conflict Point*)
A conflict point is a cell or an edge, that two different agents share on their planned path.

Formally, we define the set of all conflict points between two agents Cop.

$$Cop(p^{a_1}, p^{a_2}) = \{c \mid \forall t_1, t_2 \in \mathbb{N}, \\ c \in Pos_e(a_1, p^{a_1}, t_1), c \in Pos_e(a_2, p^{a_2}, t_2)\}$$

Take the purple agent in figure 4. If it malfunctions at its starting position, the number of steps it takes to reach any of the cells on its path increase. However, to determine triviality we are only interested in the number the steps the purple agent takes to get to the conflict point. This is because the conflict point is the only cell at which the malfunction can actually result in a collision with the red agent.

We can see, that when it comes to conflict points we are often interested in the number of steps an agent takes to reach it. Let us now discuss this aspect in more detail.

4.3 Deltas

Deltas record how many steps agents need to take to reach a conflict point.

Importantly, there can be multiple deltas for a given agent and a given conflict point. This is because an agent might enter the cell multiple times, or stay on the cell for more than one step. As a result, we will define deltas of an agent to a conflict point as a set. Each element is one of the possible number of steps the agent takes to reach the cell.

Definition 14 (*Delta / δ*)
A delta is the number of steps an agent a takes of its plan p^a, to reach a conflict point c.

Formally, we define the set of all possible deltas of agent a to c as

$$\Delta_a = \{t \mid c \in Pos_e(a, p^a, t)\}$$

In our notation, single values of Δ_a will be referred to using δ_a. This means $\delta_a \in \Delta_a$ is always true.

Let us go back again to the example of figure 4. We have already said, that the red cell is a conflict point. Now, given our plans for the purple and red agent to continuously go forward, we can also see that the deltas of either agent to reach the conflict point are $\delta_{purple} = 2$ and $\delta_{red} = 3$.

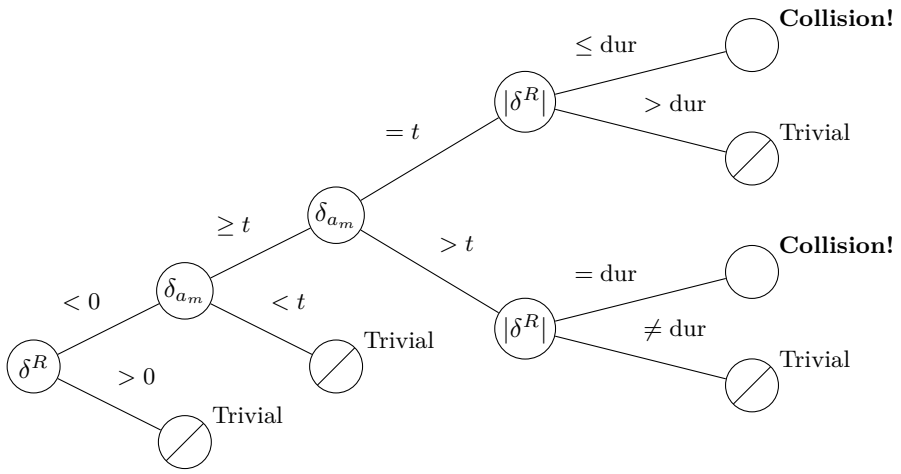


Figure 5: Decision tree showing the process of determining triviality using deltas. Note, that $|\delta^R|$ refers to the absolute value of δ^R.

4.3.1 Relative Delta

We often compare the deltas of two agents by looking at their difference. For ease of use, we will define the shorthand of 'relative deltas'.

Definition 15 (*Relative Delta* / δ^R)
Given a pair of deltas, δ_{a_1} and δ_{a_2}, where a_1 is a malfunctioning agent, δ^R records the difference.

$$\delta^R = \delta_{a_1} - \delta_{a_2}$$

Note, that δ_{a_1} of malfunctioning agent a_1 is always the minuend.

We use δ^R when looking at a given pair of deltas. For ease of use, we have δ^R implicitly record their differences as defined.

4.4 Determining Triviality Using Deltas

Let us now use conflict points, deltas and relative deltas to determine triviality.

For any one case, we will be looking at...

1. a malfunction $m = (a_m, dur, t)$ of an agent a_m,
2. another agent a_o,
3. a conflict point $c \in Cop(p^{a_m}, p^{a_o})$,
4. a delta for the malfunctioning agent $\delta_{a_m} \in \{t \mid c \in Pos_e(a_m, p^{a_m}, t)\}$,
5. a delta for the other agent $\delta_{a_o} \in \{t \mid c \in Pos_e(a_o, p^{a_o}, t)\}$,
6. a relative delta $\delta^R = \delta_{a_m} - \delta_{a_o}$, and its absolute value $|\delta^R|$.

The process by which we determine triviality can be seen in figure 5.

1. We check $\delta^R > 0$:
    - If true, the case is trivial.
    - Else, continue with 2 below...
2. We check $\delta_{a_1} < t$:
    - If true, the case is trivial.
    - Else, continue with 3 below...
3. We check $\delta_{a_1} = t$:
    - If true, continue with 4 below...
    - Else, continue with 5 below...
4. We check $|\delta^R| > dur$:
    - If true, the case is trivial.
    - Else, the case is *not trivial* and results in a **collision**.
5. We check $|\delta^R| \neq dur$:
    - If true, the case is trivial.
    - Else, the case is *not trivial* and results in a **collision**.

We will refer to the five conditions above as $tric_1, tric_2, \ldots, tric_5$.

First Condition: $tric_1 = \delta^R > 0$

With the first condition we separate out cases in which the malfunctioning agent would have reached the conflict point *after* the other train. The malfunction only makes the gap by which the malfunctioning agent misses the other agent by longer. This means, if for our case $\delta^R > 0$ holds, the case is trivial.

Second Condition: $tric_2 = \delta_1 < t$

The second condition filters out trivial cases in which the malfunctioning agent has already gone past the conflict point. The malfunctioning agent takes δ_1 steps to reach the conflict point. Should the malfunction happen at a later point t, we know the case is trivial.

Third Condition: $tric_3 = \delta_{a_1} = t$

The third Condition does not allow us to immediately decide if the case is trivial or not, unlike the previous two conditions. In essence we are asking if the agent is malfunctioning *on* or *before* the conflict point.

If $\delta_1 = t$, we know the malfunction is happening *on* the conflict point. If $\delta_1 > t$, we know it is happening *before* the conflict point. All cases where $\delta_1 < t$ were already separated out by $tric_2$.

Fourth Condition: $tric_4 = |\delta^R| > dur$

If the malfunction occurs *on* the conflict point, condition $tric_4$ checks if it lasts as long as it needs to for the other train to 'catch up'. $|\delta^R|$ records the gap of steps between the two agents to reach the conflict point. If a_1 stays on the conflict point for at least $|\delta^R|$ steps, the other agent will be able to take enough steps to reach the point as well. A collision results. Otherwise the case is trivial.

Fifth Condition: $tric_5 = |\delta^R| \neq dur$

If the malfunction occurs *before* the conflict point, condition $tric_5$ checks if the malfunction lasts just as long as it needs to, to reduce the relative delta δ^R between the agents to 0. This happens when the malfunction is as long as $|\delta^R|$.

When $|\delta^R| = dur$, both agents will reach the conflict point at the same time, resulting in a collision.

With this we arrive at a formalization of how we can functionally determine triviality using deltas.

Definition 16 (*Simple Triviality Function*)
For...

- a malfunction $m = (a_m, dur, t)$ of an agent a_m,
- another agent a_o,
- a conflict point $c \in Cop(p^{a_m}, p^{a_o})$,
- a delta for the malfunctioning agent $\delta_{a_m} \in \{t \mid c \in Pos_e(a_m, p^{a_m}, t)\}$,
- a delta for the other agent $\delta_{a_o} \in \{t \mid c \in Pos_e(a_o, p^{a_o}, t)\}$,
- a relative delta $\delta^R = \delta_{a_m} - \delta_{a_o}$,

...we define ...

- $tric_1 := \delta^R > 0$,
- $tric_2 := \delta_{a_m} < t$,
- $tric_3 := \delta_{a_m} = t$,
- $tric_4 := |\delta^R| > dur$,
- $tric_5 := |\delta^R| \neq dur$.

Simple triviality tri^R_δ is then defined as follows.

$$tri^S_\delta(m, a_o, c, \delta_{a_m}, \delta_{a_o}) = \begin{cases} 1, \text{ if } tric_1 \vee \\ \quad (\neg tric_1 \wedge tric_2) \vee \\ \quad (\neg tric_1 \wedge \neg tric_2 \wedge tric_3 \wedge tric_4) \vee \\ \quad (\neg tric_1 \wedge \neg tric_2 \wedge \neg tric_3 \wedge tric_5), \\ 0, \text{ else} \end{cases}$$

The simple triviality function lets us check if a malfunction m of agent a_m is trivial in respect to one other agent a_o. To check if a malfunction is trivial as a whole, we need to check the triviality of m against all other agents in the environment.

Definition 17 (*Complete Triviality Function*)
Complete triviality is defined as

$$tri_{\delta}(m) = \begin{cases} 1, \text{ if } \forall a_o \in Agents/a_m. \\ \quad \forall c \in Cop(p^{a_m}, p^{a_o}). \\ \quad \forall \delta_{a_1} \in \{t \mid Pos_e(a_1, p^{a_1}, t) = c\}. \\ \quad \forall \delta_{a_2} \in \{t \mid Pos_e(a_1, p^{a_1}, t) = c\}. \\ \quad tri_{\delta}^{S}(m, a_o, c, \delta_{a_m}, \delta_{a_o}) = 1, \\ 0, \text{ else} \end{cases}$$

where $Agents$ is the set of all agents in the environment,
a_m is the malfunctioning agent of malfunction $m = (a_m, dur, t)$
and all p^a are elements of the active environment plan P_{Agents}.

If $tri_{\delta}(m)$ holds true, the malfunction $m = (a_m, dur, t)$ is trivial. This means, we can proceed with our old environment plan P_{Agents}, by simply updating the actions of the malfunctioning agent a_m to include wait commands for its malfunction, $p'^{a_m} = ins(p^{a_m}, dur, t)$. The resulting new plan then looks like this:

$$P'_{Agents} = P_{Agents}/\{p^{a_m}\} \cup \{p'_{a_m}\}.$$

4.5 Examples

4.5.1 First Example

Recalling the example that motivated this section, seen in figure 4, we will observe the effects of adding a malfunction to the purple agent.

Let us again suppose our original solution

- $p^{a_{red}} = (1, 1, 1, 1, \dots)$
- $p^{a_{purple}} = (1, 1, 1, 1)$.

We look at the conflict point $c = (6, 5) \in Cop(a_{red}, a_{purple})$. This is the red cell in figure 4. The deltas for agents a_0 and a_1 are

- $\delta_{a_{purple}} = 2$
- $\delta_{a_{red}} = 4$.

Let us suppose a malfunction $m = (a_{purple}, 2, 0)$. As a result, the relative delta in this case is $\delta^R = 2 - 4 = -2$.

To determine $tri^R_\delta(m)$, we only have to determine $tri^R_\delta(m, a_{red}, c, \delta_{a_m}, \delta_{a_o})$. Let us go through the conditions $tric_1, \dots, tric_5$.

First Condition

Inserting $\delta^R = -2$ into $tric_1 = \delta^R > 0$ gives us $-2 > 0$. This is not true, so the malfunction is happening for the train that reaches the conflict point first. We move on to checking the next condition, $tric_2$.

Second Condition

Inserting $\delta_{a_{purple}} = 2$ and $t = 0$ into $tric_2 = \delta_{a_{purple}} < t$ gives us $2 < 0$. This is not true, so the malfunction happens before or on the conflict point. We move on to checking the next condition, $tric_3$.

Third Condition

Inserting $\delta_{a_{purple}} = 2$ and $t = 0$ into $tric_3 = \delta_{a_{purple}} = t$ gives us $2 = 0$. This is not true, so the malfunction happens *before*, not *on* the conflict point. As a result, we move on to checking condition $tric_5$.

Fifth Condition

Inserting $\delta^R = -2$ and $dur = 2$ into $tric_5 = |\delta^R| \neq dur$ gives us $2 \neq 2$. This is not true, meaning the malfunction is not trivial and results in a collision!

4.5.2 Second Example

Let us now have a look at the pink agent a_{pink}. In figure 6 we can see the relevant part of the environment highlighted. Unlike in the first example, the two agents a_{red} and a_{pink} have multiple conflict points between them. Here we are looking at the edge highlighted in red $c = \{(12, 6), (12, 7)\} \in Cop(a_{red}, a_{purple})$.

The deltas of either agent to c are $\delta_{a_{pink}} = 2$ and $\delta_{a_{red}} = 12$. Supposing a_{pink} malfunctions, the relative delta becomes $\delta^R = -10$.

Let us look at a malfunction $m = (a_{pink}, 11, 1)$. In 7 we can see the reasoning of why this case is trivial by following the colored path.

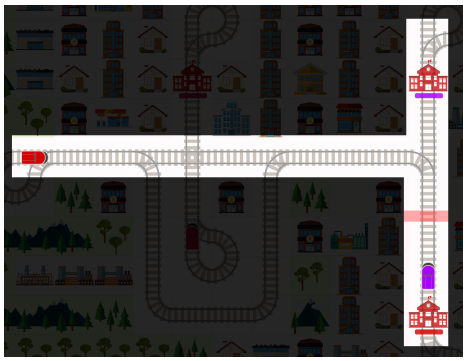


Figure 6: Partial example Flatland environment, with a conflict point marked in red

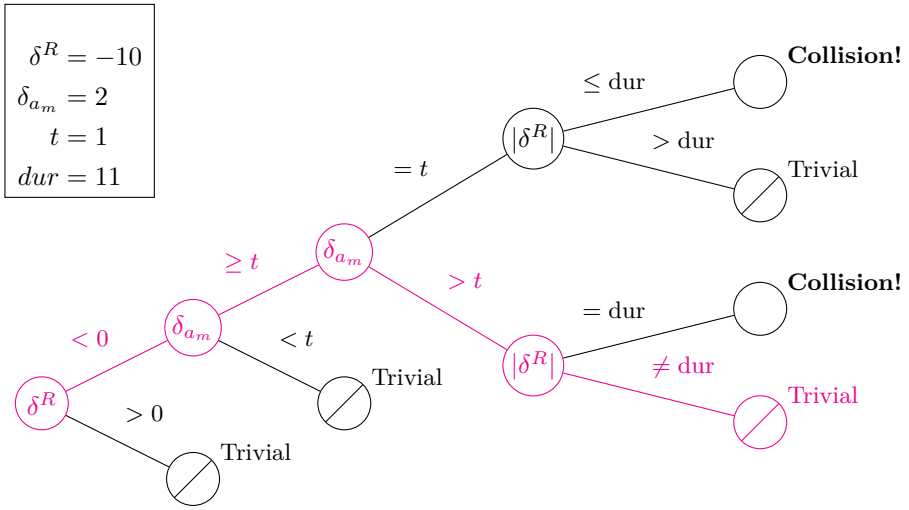


Figure 7: Decision tree showing the process of determining triviality of second example case

5 Replanning

If we determine that a malfunction is not trivial, we move on to replanning. In practice, this means when a malfunction is created on the simulation side (Flatland) and we have determined it to not be trivial on the planning side (ASP), the planning side is responsible for creating a new environment solution. In ASP, planning of any new solution has to include a base set of facts made up of...

1. all base facts about the environment, meaning all cell facts and original agent facts about start and end positions,
2. all action facts of actions that have already been executed in the simulation, meaning all actions of all agents before the malfunction,
3. the wait action facts inserted for the malfunctioning agent.

We can choose to either modify the previously active plan, or to come up with a new plan based on just the base set of facts. In either case, the new environment plan is supposed to get all agents from their original starting point to their destination, just like the plan before. No changes are ever made to the original starting points of the agents. This means, the new plan and the previous plan will always have all actions in common up until the malfunction. Action facts after the malfunction have to be generated using a given replanning strategy.

In this section we will present three replanning strategies. Sections 5.1 and 5.3 will cover strategies that take advantage of the facts from the previously active plan, adapting it, while section 5.2 will discuss an approach that creates a new plan from just the base set of facts.

These approaches are in no way meant to be exhaustive, but instead serve to present some general ideas of how replanning can be done, what general trade-offs can be observed and which areas warrant further exploration.

In contrast to the previous section on triviality, our discussion of replanning will contextualize changes made to agent plans within environment solutions. Partly, this is because environment solutions are the output of replanning, so it makes sense to work with them directly. It is also owing to how changing an agent's plan can have implications to the rest of the agents, unlike in triviality checking. There, properties about plans, like the relative deltas between all agents being non-zero, could be assumed. No plans are changed when checking for triviality, so these properties always held. In replanning, since changing the plan of one agent can cascade into having to change, in the worst case, the plans of all other agents, we need to broaden our scope.

5.1 Naive Replanning

The most straightforward way to guarantee a valid solution after a malfunction $m = (a_m, dur, t)$ is to also have the unaffected agents wait out the malfunction. To do so, we take the original environment solution $P_{Agents} = \{p^{a_1}, \ldots, p^{a_m}, \ldots, p^{a_i}\}$ to create a new solution,

$$P'_{Agents} = \{p'^a \mid \forall p^a \in P_{Agents}, p'^a = ins(p^a, dur, t)\}.$$

This works because making all agents wait out the malfunction means all agents will be able to start moving again from the position they would have been in had the malfunction not occurred. If the agents then execute the old plan from where they left off as if no malfunction *had* occurred, they will naturally all get to their destination like before.

The clear upside of this approach is its simplicity. It is possible however, that unnecessary delays are caused. A train malfunctioning on one side of the environment will always make all other trains halt, no matter if these trains are on the other side of the environment and do not share any conflict points with the malfunctioning train.

Naive replanning guarantees that we will always have a valid plan to switch to in case of a non-trivial malfunctions. Due to the limitations noted above, we might find the resulting solution to be unfavorable. Let us discuss how we can get more options.

5.2 Redo Replanning

A very direct approach to finding multiple new solutions is to go through the process of solving the environment using the same ASP encoding also used for the original plan. Instead of only passing the environment facts to the the encoding to solve, we also constrain the output models, so that they always contain the action facts from the base set of facts that need to be part of any new plan, described at the start of this section.

This way of replanning can be costly, because it explores all solution options the encoding can generate. In return we have the potential of being offered more solutions to choose from.

5.3 Wait Replanning

Given a non trivial malfunction $m = (a_m, dur, t)$, we take the previous environment solution $P_{Agents} = \{p_{a_1}, \ldots, p_{a_m}, \ldots, p_{a_i}\}$ as a base to come up with a new solution $P'_{Agents} = \{p'_{a_1}, \ldots, p'_{a_m}, \ldots, p'_{a_i}\}$. The malfunctioning agent gets all necessary wait actions added, $p'_{a_m} = ins(p_{a_m}, dur, t)$, but unlike in naive replanning, the other agents are allowed to wait for *differing* durations, $\forall p_a \in P_{Agents}/\{p_{a_m}\}.\exists t_a.p'_a = ins(p_a, dur, t_a)$. This way we limit replanning to just adding wait commands.

If the malfunction has the duration dur, we do not need to try adding more than dur wait commands to any of the other agents to be guaranteed a valid plan. This is because, if we add exactly dur wait command to every agent, the resulting plan is equivalent to that of naive replanning, which we know to be valid.

Assuming the number of agents in the environment is $|Agents|$ and that we experience one malfunction of length dur, we are left with $(dur + 1)^{(|Agents|-1)}$ different distributions of wait actions to try.

Constricting replanning to just adding wait actions has the advantage that the size of the environment does not have an impact on the search space. The only relevant variables are malfunction length and number of agents. However, trying out all $(dur+1)^{(|Agents|-1)}$ distributions of wait actions is still only feasible for a relatively small number of agents, because of the exponential nature by which options grow when we add agents to the environment.

5.4 Replanning Requirements

As we have seen in our discussion of naive replanning, coming up with a new valid solution can be achieved relatively easily and cheaply. However, the resulting new plan might not be to our liking. We have looked at other approaches that provide more plans to chose from. In this section we will look at the requirements we can set, by which we evaluate these new plans.

5.4.1 Hard Requirements

Hard requirements are necessary conditions our new solution P'_{Agents} will have to fulfill.

For one, the new solution has to include all the action commands from the base set of facts discussed at the start of this section. This means, the new solution should have all the same agent actions of the previous plan up to the malfunction start and the required wait actions for the malfunctioning agent.

Secondly, the new solution has to be valid. This means, all agents reach their destinations without collisions.

5.4.2 Soft Requirements

Soft requirements are set by the user. These might be finding a solution with the least amount of time taken, a solution with the least conflict points or with the most similar routing to the previous plan. The user might also chose a combination of these requirements.

Naive replanning for example fulfills the hard requirements for replanning, but is likely outdone by other approaches when it comes to the soft requirement of providing, say, the fastest solution.

A soft requirement always tries to minimize, or in rarer cases maximize, some property of a solution. We will later discuss how we can express this in ASP.

If we do not search through all solutions for the environment, soft requirements are 'best effort', meaning we might end up with a solution that is good enough, but not guaranteed to be the best. Since searching through all solutions can be costly, we might only spend a set amount of time, or look at a set number of models before deciding to take the one suiting our soft requirements best.

Minimizing plan length and deviation from the original plan when replanning in particular has already been a topic of academic discussion, such as in Spliet *et al.* (2014)[9].

6 Implementation

In this section we will discuss a possible implementation of the ideas discussed in the paper[4].

6.1 Python Framework

6.1.1 Overview

We work with Flatland and ASP in Python.

The flow of the Python framework stays the same as shown in 3.3.3. To recap, the steps look like this.

1. **Build** the environment (using Python & Flatland)
2. **Solve** the environment (using ASP)
3. **Simulate** the environment (using Flatland)
4. *In case of malfunction:* **Check** triviality (using ASP)
5. *In case of non-triviality:* **Replan** the solution (using ASP)

6.1.2 ASP Knowledge Base

To address the possible need to access previous solutions when replanning, as well as to simplify the storage of all things related to ASP, the implementation introduces the "ASP Knowledge Base".

An object of this class is initialized when solving any environment and serves to store ASP facts. We use it to store environment facts, the active encoding and all solutions we want to keep hold of.

The list of all solutions is updated when solving the environment for the first time and for every time we replan due to a malfunction. In those cases we are free to append as many new solutions to the list as we want. The active solution is always the last item in the list, meaning if we update the list, we also change the active plan. The other solutions are stored so that they can be used for replanning.

6.1.3 Property Agent

Taking our ASP knowledge of the environment and our solution from our knowledge base, we are able to derive properties such as the positions of agents. This way, we can model properties we have defined previously, such as positions (def. 3 & 5), or conflict points (def. 13).

In the implementation we can do this by introducing a "property agent". The property agent takes ASP facts and uses them to derive properties. The property agent is given an ASP knowledge base and a property to solve for. Since most properties depend on other properties, the property agent resolves the dependencies to get to the property. Figure 8 shows a dependency diagram going from the base properties contained in the knowledge base at the bottom to triviality at the top.

[4]A work in progress fork can be found on GitHub

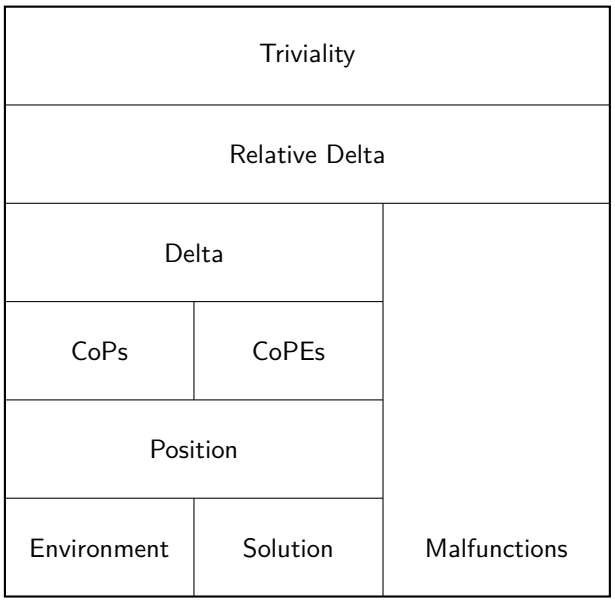


Figure 8: Graphic showing the property dependencies of finding triviality using deltas

6.2 ASP Encoding

6.2.1 Triviality

We can look at triviality as a property, which can be derived from a set of base ASP facts. This base set are the facts of the Flatland environment, the active solution and a malfunction.

The property agent can use these facts to determine triviality. It does so by using deltas, with the same logic that was discussed in section 4.

However, it is important to note that the only parts that are fixed here are the base set of facts and the top level property to be determined, triviality. All intermediate layers of properties can be replaced, if a different approach is used.

For every property we create a new encoding file. The file for non triviality could look like this:

```
1  non_trivial :-
2      malfunction(train(ID1),Dur,T), delta(train(ID1), Dt1),
3      relative_delta(train(ID1),train(ID2),Dr),
4      Dr < 0,           % malfunctioning train is ahead
5      T = Dt1,          % malfunction is early enough
6      Dr <= Dur.        % malfunction is long enough
7
8  non_trivial :-
9      malfunction(train(ID1),Dur,T), delta(train(ID1), Dt1),
10     relative_delta(train(ID1),train(ID2),Dr),
11     Dr < 0,           % malfunctioning train is ahead
12     T < Dt1,          % malfunction is early enough
13     Dr = Dur.         % malfunction is long enough
```

When the property agent resolves the dependencies of triviality, it loads all necessary files. That includes the one above, along with those of all prerequisite property files. Let us have a look at another example of a property file.

6.2.2 Deltas

As a subordinate layer to triviality, deltas also need to be determined. Accordingly, the property agent will load the relevant file.

The file responsible for providing the 'delta' property could look like this:

```
delta(train(ID),(X,Y),D) :-
    conflict_point(train(ID),_,(X,Y)),
    position(train(ID),(X,Y),_,D).
delta(train(ID),(X,Y),D) :-
    conflict_point(_,train(ID),(X,Y)),
    position(train(ID),(X,Y),_,D).
```

6.2.3 Replanning

Once the result of the property agent tells us a malfunction is not trivial, we can move on to replanning.

There are multiple ways to proceed algorithmically, depending on the strategy employed. The input will always be the ASP knowledge base, the malfunction and optionally one or more soft requirements. In implementation we can interpret the provided input in whatever way we see fit to find a new solution.

The replanning function returns at least one solution, but can return more. If we provided a soft requirement, the list that is returned will have the solution with the best fit at the end. This list is appended to the ASP knowledge base's list of solutions. The last solution in the list is then taken as the new active solution.

6.2.4 Soft Requirements

As discussed in section 5.4.2, soft requirements come in the form of minimize statements, or in some cases, maximize statement.

As an example, the soft requirement property file looking for the least number of actions in a solution could look like this:

```
#minimize{ 1,ID,Action,Time : action(train(ID), Action, Time) }.
```

Passing this on to the replanning step results in the solution with the least number of actions, among all solutions returned by replanning, to become the new active solution.

7 Discussion

7.1 Machine Learning and ASP

The foundational Flatland paper puts a large emphasis on its approach to the VRSP using machine learning approaches such as reinforcement learning. This approach is synchronous, meaning agents find their path through the environment *as* they are moving. This is fundamentally different to how we approach the VRSP in Flatland.

Unlike with ML, our approach using ASP is asynchronous. This means we determine a solution before even one agent has started moving. This plays into the classification of the problem too. In this paper, we have talked about tackling the VRSP.

The term '*Real Time* Vehicle Rescheduling Problem' (RTVRSP) comes up in literature as well [10, 11]. However, we classify the problem covered here to still be the VRSP because both planning and replanning happen not simultaneously to simulation, but asynchronously. Plans and replans are made in advance before being executed, not in real time.

7.2 Concerning Replanning

The field of replanning is hard to do justice within the scope of one paper. The problem itself can be framed in the same terms as planning, like we saw in redo replanning of section 5.2. As a result, the depth and complexity can be immense. The idea in the paper was thus to offer the general ideas and rules to better understand the problem at a high level.

7.2.1 Defining a Scope

In our definition of scope for replanning, we stated that any changes made to the agent plans from the old environment solution would be contextualized within a new environment solution. In practical terms our output for replanning will always be a new environment solution. For the purposes of discussion, the scope within the environment could be reduced the same way it was for checking triviality. When replanning changes an agent's plan, we can limit our scope to just look at other agents that are possibly affected by this change. These are all agents the agent with the changed plan shares at least one conflict point with. Just like before, we can proceed agent by agent, conflict point by conflict point.

Triviality checking only had to preserve the property of an environment plan being collision free. The property of all agents reaching their destinations was provided by the valid plan. If in replanning we make changes to the movement actions of an agent plan, making sure all agents still get to their destination would need to be included in the scope.

7.2.2 Reducing the Search Space

In the discussion of wait replanning (sec. 5.3), we touched on reducing the search space by only allowing for wait actions to be added.

Of course, there are other ways of limiting the search space left to explore. These include limiting replanning to smaller sets of agents, or to smaller parts of the environment.

8 Conclusion

In this paper we have introduced a response framework addressing unforeseen train malfunctions in simulated railway environments. The framework makes sure, that an active plan in the railway environment stays valid when a malfunction occurs. It does so in two steps.

First, motivated by wanting to avoid costly replanning any time a malfunction occurs, the necessity of replanning is determined. This is based on a property we call triviality. To this end, we examined the concepts of conflict points, deltas and relative deltas, with which triviality can be determined.

Second, if deemed necessary in the fist step, the response moves on to replanning. For replanning we covered different approaches and looked at their merits and shortcomings, motivating us to formalize the idea of soft requirements on replanning strategies.

Further, we had a look at a possible implementation. In the framework's spirit of extensibility, the implementation tries to offer a solid, but malleable base for future extension, change and experimentation.

This paper hopes to meaningfully contribute to the work already done by the Knowledge Representation and Reasoning Group at the University of Potsdam on the vehicle rescheduling problem (VRSP) in Flatland.

8.1 Further Work

Both steps of the framework expect an output of a certain type. The first step expects a boolean, the second a list of plans. However, the method used to determine these outputs is exchangeable. Future research into new approaches for determining triviality and replanning is encouraged.

Further in-depth exploration is also especially pressing in the field of replanning. Here, the formalization of more advanced soft requirements and their effects, as well as the extension of the framework's planning step by integrating a logic, which chooses a fitting replanning strategy, can be looked at.

An especially important extension of the work done here would be the possibility of covering multiple malfunctions happening at the same time. Right now, multiple malfunctions are only covered if one malfunctions happens after the last has finished. As part of this research direction, new approaches to determine triviality besides delta analysis might also very well be worth examining.

Declaration of originality

I hereby certify that the thesis I am submitting is entirely my own original work except where otherwise indicated. I am aware of the University's regulations concerning plagiarism, including those regulations concerning disciplinary actions that may result from plagiarism. Any use of the works of any other author, in any form, is properly acknowledged at their point of use.

Eigenständigkeitserklärung

Hiermit bestätige ich, dass die von mir eingereichte Dissertation vollständig meine eigene Originalarbeit ist, sofern nicht anders angegeben. Mir sind die Vorschriften der Universität bezüglich Plagiaten bekannt, einschließlich der Vorschriften bezüglich disziplinarischer Maßnahmen, die sich aus einem Plagiat ergeben können. Jegliche Verwendung der Werke anderer Autoren, in welcher Form auch immer, wird an der Verwendungsstelle ordnungsgemäß anerkannt.



Signature/*Unterschrift*

Frankfurt a.M., 7.12.2025

Place/*Ort*, Date/*Datum*

References

[1] Sharada Mohanty, Erik Nygren, Florian Laurent, Manuel Schneider, Christian Scheller, Nilabha Bhattacharya, Jeremy Watson, Adrian Egli, Christian Eichenberger, Christian Baumberger, et al. "Flatland-rl: Multi-agent reinforcement learning on trains". In: *arXiv preprint arXiv:2012.05893* (2020).

[2] Florian Laurent, Manuel Schneider, Christian Scheller, Jeremy Watson, Jiaoyang Li, Zhe Chen, Yi Zheng, Shao-Hung Chan, Konstantin Makhnev, Oleg Svidchenko, et al. "Flatland competition 2020: MAPF and MARL for efficient train coordination on a grid world". In: *NeurIPS 2020 Competition and Demonstration Track*. PMLR. 2021, pp. 275–301.

[3] Richard Hájek, Ladislav Malecek, Matouš Meleckỳ, and Jaroslava Schovancová. "2021 Flatland Challenge, Team Learn". In: ().

[4] Jiaoyang Li, Zhe Chen, Yi Zheng, Shao-Hung Chan, Daniel Harabor, Peter J Stuckey, Hang Ma, and Sven Koenig. "Scalable rail planning and replanning: Winning the 2020 flatland challenge". In: *Proceedings of the international conference on automated planning and scheduling*. Vol. 31. 2021, pp. 477–485.

[5] Vladimir Lifschitz. *Answer set programming*. Vol. 3. Springer Cham, 2019.

[6] Martin Gebser, Roland Kaminski, Benjamin Kaufmann, and Torsten Schaub. "Multi-shot ASP solving with clingo". In: *CoRR* abs/1705.09811 (2017).

[7] Jing-Quan Li, Pitu B Mirchandani, and Denis Borenstein. "The vehicle rescheduling problem: Model and algorithms". In: *Networks: An International Journal* 50.3 (2007), pp. 211–229.

[8] Jing-Quan Li, Denis Borenstein, and Pitu B Mirchandani. "A decision support system for the single-depot vehicle rescheduling problem". In: *Computers & Operations Research* 34.4 (2007), pp. 1008–1032.

[9] Remy Spliet, Adriana F Gabor, and Rommert Dekker. "The vehicle rescheduling problem". In: *Computers & Operations Research* 43 (2014), pp. 129–136.

[10] Monize Sâmara Visentini, Denis Borenstein, Jing-Quan Li, and Pitu B Mirchandani. "Review of real-time vehicle schedule recovery methods in transportation services". In: *Journal of Scheduling* 17.6 (2014), pp. 541–567.

[11] Jing-Quan Li, Pitu B Mirchandani, and Denis Borenstein. "A Lagrangian heuristic for the real-time vehicle rescheduling problem". In: *Transportation Research Part E: Logistics and Transportation Review* 45.3 (2009), pp. 419–433.